Confluent Platform SQL Deployment via Flink SQL Client — Scalar UDFs

> This example deploys **two scalar UDFs** — `celsius_to_fahrenheit` and `fahrenheit_to_celsius` — by submitting SQL statements through the **Flink SQL Client** running directly on the JobManager pod. The same two UDFs are provided in **two parallel implementations**: a **Java** path (uber JAR built from examples/scalar_udf/java/) and a **Python / PyFlink** path (source files in examples/scalar_udf/python/src/). Each path drives two independent long-running Flink jobs in the same session cluster, each reading from a Kafka source topic, applying the scalar conversion, and writing to a Kafka sink topic.

Table of Contents

- 1.0 Overview
  - 1.1 How this differs from the other deployment paths
- 2.0 How it works
  - 2.1 UDF delivery
    - 2.1.1 Java — JAR via `kubectl exec`
    - 2.1.2 Python — custom `cp-flink-python` image
  - 2.2 Kafka connector
  - 2.3 Statement flow
    - 2.3.1 Java statement flow
    - 2.3.2 Python statement flow (deltas from the Java flow)
- 3.0 Prerequisites
- 4.0 How to run
  - 4.1 Deploy — Java
  - 4.2 Deploy — Python
  - 4.3 Monitor
  - 4.4 Tear down
- 5.0 Resources

1.0 Overview

1.1 How this differs from the other deployment paths

Aspect	CP — Java (this example)	CP — Python (this example)	Confluent Cloud Terraform
Where it runs	Confluent Platform on Minikube	Confluent Platform on Minikube	Confluent Cloud
How SQL is submitted	`sql-client.sh -f` on the JobManager pod	`sql-client.sh -f` on the JobManager pod	`confluent_flink_statement` Terraform resources
UDF delivery	`kubectl exec` copies uber JAR onto Flink	Custom `cp-flink-python` image with venv	`confluent_flink_artifact` (uploaded to CC)

Aspect	CP — Java (this example)	CP — Python (this example)	Confluent Cloud Terraform
	pods	+ `.py` files baked in	
Language / runtime	Java 21, loaded from a file-local JAR	PyFlink (Python 3.11 + `apache-flink` wheel from `uv.lock`)	Java 21
Entry point	`make deploy-cp-scalar-udf`	`make deploy-cp-scalar-udf-python`	`make deploy-cc-scalar-udf`
Requires code build	✅ Java + Gradle (UDF JAR)	✅ Docker build of `cp-flink-python` image (bakes in venv + `.py` files)	✅ Java + Gradle (UDF JAR)
Statement lifecycle	Managed by Flink session cluster	Managed by Flink session cluster	Managed by Terraform state
Same SQL shape?	`CREATE FUNCTION … USING JAR 'file:///…/scalar-udf.jar'`	`CREATE FUNCTION … LANGUAGE PYTHON` + `SET 'python.executable'` / `'python.files'`	`CREATE FUNCTION … USING JAR 'confluent-artifact://…'`

2.0 How it works

2.1 UDF delivery

On Confluent Platform there is no artifact store — everything the Flink runtime needs must be physically present on the JobManager and TaskManager pods before the `CREATE FUNCTION` statement is executed. The Java and Python paths solve this differently.

2.1.1 Java — JAR via `kubectl exec`

The deploy script copies the uber JAR (built from examples/scalar_udf/java/) to `/opt/flink/usrlib/scalar-udf.jar` on every JobManager and TaskManager pod using `kubectl exec`. Both `CREATE FUNCTION ... USING JAR` statements (`celsius_to_fahrenheit` and `fahrenheit_to_celsius`) reference this same pod-local path, since both scalar UDFs are bundled in the same uber JAR.

2.1.2 Python — custom `cp-flink-python` image

PyFlink cannot load a `.py` file via `USING JAR`; the Python interpreter, the `apache-flink` wheel, and the UDF source files must already be on every JM/TM pod before SQL is submitted. This example bakes them all into a custom Flink image (k8s/images/cp-flink-python/Dockerfile) built on top of `confluentinc/cp-flink`:

1. **Python 3.11** is installed into the base image via `apt`.

2. **A `uv`-built venv** is staged at `/opt/flink/python-udf/.venv/` containing the `apache-flink` wheel, resolved reproducibly from examples/scalar_udf/python/uv.lock.
3. **The UDF source files** (`celsius_to_fahrenheit.py`, `fahrenheit_to_celsius.py`) are copied to `/opt/flink/python-udf/`.

The deploy script then issues three `SET` statements at the top of its SQL session to point PyFlink at those baked-in paths:

```
SET 'python.executable'        = '/opt/flink/python-udf/.venv/bin/python';
SET 'python.client.executable' = '/opt/flink/python-udf/.venv/bin/python';
SET 'python.files'             = '/opt/flink/python-
udf/celsius_to_fahrenheit.py,
                                 /opt/flink/python-
udf/fahrenheit_to_celsius.py';
```

and registers each function with `CREATE FUNCTION … AS '<module>.<function>' LANGUAGE PYTHON` (no `USING JAR` clause).

2.2 Kafka connector

The CP Flink base image does not include the Kafka SQL connector. The FlinkDeployment uses two **init containers** to assemble `/opt/flink/lib/` at pod startup:

1. **copy-stock-lib** — copies the stock Flink lib JARs from the image into a shared `emptyDir` volume
2. **fetch-kafka-connector** — downloads `flink-sql-connector-kafka` from Maven Central into the same volume

The main Flink container then mounts this volume at `/opt/flink/lib/`, so the Kafka connector is on the classpath when the JVM starts. This is identical for the Java and Python deployments; the Python image simply adds Python 3.11 + the venv on top.

2.3 Statement flow

Both paths pre-create **all four Kafka topics**, then execute all SQL for **both** scalar UDFs in a single `sql-client.sh -f` session on the JobManager pod.

2.3.1 Java statement flow

Both UDFs are bundled in the same uber JAR, so a single pod-local JAR path feeds both `CREATE FUNCTION` statements.

```
┌──────────────────────────────────────────────────────────────────────────────
──┐
│  Pre-step: kafka-topics --create celsius_reading, celsius_to_fahrenheit,
│
│                                  fahrenheit_reading,
fahrenheit_to_celsius │
│
```

```
  │
  │  ── UDF 1: CelsiusToFahrenheit ──
  │
  │  Step  1:  DROP TABLE IF EXISTS celsius_reading                → OK
  │
  │  Step  2:  CREATE TABLE celsius_reading (... WITH kafka ...)   → OK
  │
  │  Step  3:  INSERT INTO celsius_reading VALUES (sample data)    →
submitted │
  │  Step  4:  DROP TABLE IF EXISTS celsius_to_fahrenheit          → OK
  │
  │  Step  5:  CREATE TABLE celsius_to_fahrenheit (... WITH kafka) → OK
  │
  │  Step  6:  CREATE FUNCTION celsius_to_fahrenheit               → OK
  │
  │            AS 'scalar_udf.CelsiusToFahrenheit'
  │
  │            USING JAR 'file:///opt/flink/usrlib/scalar-udf.jar'
  │
  │  Step  7:  INSERT INTO celsius_to_fahrenheit                   →
submitted │
  │            SELECT sensor_id, celsius_temperature,
  │
  │                   celsius_to_fahrenheit(celsius_temperature)
  │
  │              FROM celsius_reading
  │
  │
  │
  │  ── UDF 2: FahrenheitToCelsius ──
  │
  │  Step  8:  DROP TABLE IF EXISTS fahrenheit_reading             → OK
  │
  │  Step  9:  CREATE TABLE fahrenheit_reading (... WITH kafka)    → OK
  │
  │  Step 10:  INSERT INTO fahrenheit_reading VALUES (sample data) →
submitted │
  │  Step 11:  DROP TABLE IF EXISTS fahrenheit_to_celsius          → OK
  │
  │  Step 12:  CREATE TABLE fahrenheit_to_celsius (... WITH kafka) → OK
  │
  │  Step 13:  CREATE FUNCTION fahrenheit_to_celsius               → OK
  │
  │            AS 'scalar_udf.FahrenheitToCelsius'
  │
  │            USING JAR 'file:///opt/flink/usrlib/scalar-udf.jar'
  │
  │            -- same JAR path as Step 6, two functions, one artifact
  │
  │  Step 14:  INSERT INTO fahrenheit_to_celsius                   →
submitted │
  │            SELECT sensor_id, fahrenheit_temperature,
  │
  │                   fahrenheit_to_celsius(fahrenheit_temperature)
```

```
  │
  │              FROM fahrenheit_reading
  │
  └──────────────────────────────────────────────────────────────────────────────
 ──┘
```

Steps 7 and 14 are **two long-running streaming jobs** running side-by-side in the same Flink session cluster: Step 7 reads from `celsius_reading`, applies `celsius_to_fahrenheit`, and writes to `celsius_to_fahrenheit`; Step 14 reads from `fahrenheit_reading`, applies `fahrenheit_to_celsius`, and writes to `fahrenheit_to_celsius`.

2.3.2 Python statement flow (deltas from the Java flow)

The Python path submits the same 14 logical steps in the same order. Two differences only:

1. **Three `SET` statements prepended** to the session, pointing PyFlink at the baked-in interpreter and UDF files (see §2.1.2).
2. **`CREATE FUNCTION` uses `LANGUAGE PYTHON`** instead of `USING JAR`. Steps 6 and 13 become:

   ```sql
   -- Step 6 (Python):
   DROP FUNCTION IF EXISTS celsius_to_fahrenheit;
   CREATE FUNCTION celsius_to_fahrenheit
       AS 'celsius_to_fahrenheit.celsius_to_fahrenheit'
       LANGUAGE PYTHON;

   -- Step 13 (Python):
   DROP FUNCTION IF EXISTS fahrenheit_to_celsius;
   CREATE FUNCTION fahrenheit_to_celsius
       AS 'fahrenheit_to_celsius.fahrenheit_to_celsius'
       LANGUAGE PYTHON;
   ```

   The `AS` value is `'<module>.<function>'` — the Python module name (matching the `.py` file name from `python.files`) and the function symbol inside it.

Everything else — the four table DDLs, the sample-data `INSERT`s, and the two streaming `INSERT … SELECT` pipelines — is byte-for-byte identical to the Java path.

3.0 Prerequisites

- macOS with Homebrew or Linux with apt-get, and Docker Desktop running
- Confluent Platform and Flink stack already deployed via:

  ```
  make install-prereqs       # installs tooling (docker, kubectl,
  minikube, helm, gradle)
  make cp-up                 # Minikube → CFK Operator → Kafka + SR +
  ```

```
Connect + C3
make cp-watch              # watch pods come up (Ctrl+C when all
Running)
make flink-up              # cert-manager → Flink Operator → CMF →
Flink session cluster
make flink-status          # verify Flink pods are Running
```

- **For the Java path**: Java 21 and Gradle installed (for building the UDF JAR).
- **For the Python path**: the Flink session cluster must be running the `cp-flink-python` image with Python 3.11 + the `apache-flink` venv + the UDF `.py` files baked in. Build it into the Minikube docker daemon before deploying:

  ```
  make build-cp-flink-python-image    # docker build cp-flink-
  python:2.1.1-cp1-java21
  ```

  If `make flink-up` was run against the default `cp-flink` image, redeploy the cluster so pods pick up the Python image (see the target's output for the exact follow-up command).

4.0 How to run

All commands are run from the **project root** (where the `Makefile` lives).

4.1 Deploy — Java

A single target builds the UDF JAR, copies it to the Flink pods, and executes all SQL:

```
make deploy-cp-scalar-udf
```

Behind the scenes this runs:

Step	What it does
1	`./gradlew clean shadowJar` — builds the UDF uber JAR from `examples/scalar_udf/java/`
2	`kubectl exec` — copies the JAR to all JobManager and TaskManager pods
3	`kafka-topics --create` — pre-creates Kafka topics (`celsius_reading`, `celsius_to_fahrenheit`, `fahrenheit_reading`, `fahrenheit_to_celsius`)
4	`sql-client.sh -f` — executes all SQL statements for **both** UDFs in a single session on the JobManager pod

4.2 Deploy — Python

A single target verifies the Python image is in use, creates topics, and executes all SQL:

```
make deploy-cp-scalar-udf-python
```

Behind the scenes this runs:

Step	What it does
1	**Verify image** — `kubectl exec` checks that `/opt/flink/python-udf/.venv/bin/python` and the UDF `.py` files exist on the JobManager pod; fails fast with a rebuild hint if not
2	`kafka-topics --create` — pre-creates the same four Kafka topics as the Java path
3	`sql-client.sh -f` — executes `SET 'python.executable'` / `'python.client.executable'` / `'python.files'` plus all table DDLs, `CREATE FUNCTION … LANGUAGE PYTHON`, and streaming `INSERT`s in a single session

Note there is **no JAR build step** — the Python venv and source files are baked into the image at `make build-cp-flink-python-image` time, not at deploy time.

4.3 Monitor

Open the Flink Dashboard to see the **two running streaming jobs** (Celsius → Fahrenheit and Fahrenheit → Celsius):

```
make flink-ui              # opens http://localhost:8081
```

This works identically for both the Java and Python deployments.

4.4 Tear down

Each path has its own teardown target, which cancels running jobs via the Flink REST API, drops the two UDFs and four tables, and deletes the four Kafka topics:

```
make teardown-cp-scalar-udf            # Java path
make teardown-cp-scalar-udf-python     # Python path
```

The Kafka topic names (`celsius_reading`, `celsius_to_fahrenheit`, `fahrenheit_reading`, `fahrenheit_to_celsius`) are shared between the two paths, so only run one at a time against the same cluster — or tear the first down before deploying the second.

5.0 Resources

- Flink SQL Client
- Flink Kafka Connector
- User-defined Functions (Flink)

- Python UDFs (PyFlink)
- Create a User-Defined Function (Confluent Cloud)